

Testing — A Readable Companion to the OTP Docs

Chapter 8 · checked against OTP 29.1.1 · Sep 25, 2026

1. Why testing is where AI-written code gets caught

The previous chapters share one theme: code that looks right and passes a happy-path test, then fails under crashes, timing or load. Tests are where that gap gets closed. Only tests that deliberately cause the unhappy paths catch those bugs; tests that re-check the happy path don't.

This matters more when an assistant writes the code, because an assistant asked for tests tends to write tests that mirror the implementation: same assumptions, same blind spots. The tools below are useful exactly to the extent that they check behaviour the author didn't think about.

Erlang has four tools for this, each for a different job:

Tool	Best for	Ships with OTP
EUnit	Fast unit tests of functions and small processes	Yes
Common Test	Integration and system tests, whole applications, multiple nodes	Yes
Dialyzer	Finding type contradictions without running code	Yes
PropEr (Elixir: StreamData)	Generated inputs and randomised state-machine tests	No, a library

2. The building blocks

EUnit

A function ending in `_test()` is a test. A function ending in `_test_()` is a *generator* that returns tests, which is how you attach fixtures. `?assert`, `?assertEqual`, `?assertMatch` and `?assertError` do the checking; their underscore forms (`?_assertEqual`) build test objects for generators.

```
counter_test() ->
  {foreach,
   fun() -> {ok, P} = counter:start_link(), P end,      %% setup
   fun(P) -> gen_server:stop(P) end,                    %% cleanup
   [fun(P) -> ?_assertEqual(0, counter:get(P)) end,
    fun(P) -> ?_test(begin counter:inc(P),
                     ?assertEqual(1, counter:get(P)) end) end]}.
```

Each test runs in a separate process with a default timeout of 5 seconds.

Common Test

A suite is a module named `*_SUITE`. Configuration callbacks run at each level: `init_per_suite`, `init_per_group` and `init_per_testcase`, each with a matching `end_per_...`. Groups can run test cases in sequence, in parallel or in random order. Common Test is the right place to boot the real application and test it the way it runs in production.

Dialyzer

Dialyzer infers *success typings*: the widest types for which a function could possibly succeed. It reports only contradictions it can prove, so every warning is worth reading. The flip side, as the docs say, is that it “will sometimes not report every bug.” A clean run doesn’t mean correct code.

Property-based testing

Instead of writing examples, you state a property that must hold for all inputs and let the tool generate hundreds of cases. When one fails, the tool *shrinks* it to a minimal counterexample. The stateful variant generates random sequences of API calls against a model, which is the most effective way to find the ordering and restart bugs from earlier chapters.

3. How the runtime actually shapes your tests

Ordering gives you free synchronisation

Chapter 2’s guarantee (messages from one sender to one receiver stay in order) is a testing tool. If your test casts to a server and then calls it, the call is only handled after the cast. So a `call` works as a barrier that proves the cast was processed, with no sleeping.

Links reach into the test

Tests often start servers with `start_link`, which links the server to the test process. If the server crashes, the test process gets an exit signal. Depending on setup, that kills the test with a confusing error, or leaves a server registered under a fixed name that makes the next test fail with `already_started`.

Timeouts can skip cleanup

The EUnit docs warn that in `local` mode, if a timeout fires, “the entire fixture is abruptly terminated (without running the cleanup)”. Leftover processes and tables then leak into later tests.

4. Plausible but wrong

4.1 Sleeping to wait for async work

```
cast_test() ->
  {ok, P} = store:start_link(),
  store:put_async(P, k, v),
  timer:sleep(100),
  ?assertEqual({ok, v}, store:get(P, k)).
```

The test passes on a laptop and fails on a loaded CI machine, or it wastes 100 ms every time. Fix: remove the sleep. Because `get` is a `call` from the same process, it is handled after the cast. When the work happens in a different process, wait for a message or a monitor instead.

4.2 Tests that share a registered server

```
a_test() -> {ok, _} = cache:start_link(), ?assert(cache:ping()).
b_test() -> {ok, _} = cache:start_link(), ?assert(cache:ping()).
```

`cache` registers a name. Whether `b_test` sees `{error, {already_started, _}}` depends on whether the first server has died yet, so the suite passes or fails depending on test order. Fix: a `foreach` fixture that stops the server in cleanup, or servers that accept a name parameter so each test can use its own.

4.3 Only testing examples the author thought of

```
roundtrip_test() ->
  ?assertEqual(<<"hi">>, decode(encode(<<"hi">>))).
```

It passes, and says nothing about empty input, non-ASCII bytes, or the byte your escape table forgot. Fix: a property over generated input:

```
prop_roundtrip() ->
  ?FORALL(B, binary(), decode(encode(B)) == B).
```

`PropEr` will find the failing byte and shrink it to the smallest example.

4.4 Never testing the crash

```
worker_test() ->
  {ok, _} = my_sup:start_link(),
  ?assertEqual(ok, worker:do(job)).
```

This checks that the tree starts, not that it recovers, and recovery is what supervision is for. Fix: kill the process and check that the system comes back and behaves:

```

restart_test() ->
  {ok, _} = my_sup:start_link(),
  Old = whereis(worker),
  Ref = monitor(process, Old),
  exit(Old, kill),
  receive {'DOWN', Ref, process, Old, killed} -> ok end,
  ?assertEqual(ok, wait_for(fun() -> worker:do(job) end)).

```

Here `wait_for/1` retries a bounded number of times; the restart is asynchronous, so the test must wait for it, not sleep.

4.5 A spec that makes Dialyzer quieter, not safer

```

-spec parse(binary()) -> {ok, map()}.
parse(Bin) ->
  case json:decode(Bin) of
    M when is_map(M) -> {ok, M};
    _ -> {error, not_object}
  end.

```

The spec leaves out `{error, not_object}`. Dialyzer checks only that the spec and the inferred type overlap, and they do, so it accepts the spec and trusts it. Callers are then analysed as if errors can never happen. A caller that *does* handle `{error, _}` gets a “pattern can never match” warning, which invites someone to delete the correct error handling. Fix: write specs that describe what the function really returns, and treat narrowing specs to silence warnings as a code smell.

4.6 Tests that restate the implementation

```

price_test() ->
  ?assertEqual(100 * 0.9, price:discounted(100)).

```

If the formula is wrong, the test is wrong the same way: it was derived from the code, not from the requirement. This is the typical shape of assistant-generated tests. Fix: take expected values from the specification or a worked example (`?assertEqual(90.0, ...)`), and write properties about invariants instead: a discounted price is never negative and never exceeds the original.

5. Review checklist and sources

- ☐ No `timer:sleep/1` used for synchronisation; tests wait on calls, messages or monitors
- ☐ Every test cleans up its processes; no shared registered names between tests
- ☐ Pure functions with wide input spaces have property tests (round-trips, invariants)
- ☐ Supervision behaviour is tested by killing processes and checking recovery
- ☐ Stateful components have at least one stateful property or model-based test
- ☐ Integration tests boot the real application in Common Test
- ☐ Dialyzer runs in CI; specs describe real return values
- ☐ Expected values come from requirements, not from re-running the implementation

Sources

- EUnit User's Guide: <https://www.erlang.org/doc/apps/eunit/chapter.html>
- Common Test Basics: https://www.erlang.org/doc/apps/common_test/basics_chapter.html
- Dialyzer User's Guide: https://www.erlang.org/doc/apps/dialyzer/dialyzer_chapter.html
- PropEr: <https://proper-testing.github.io/>
- Fred Hébert, *Property-Based Testing with PropEr, Erlang, and Elixir*: <https://propertesting.com/>